

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I Sandra Sudevan-192424422 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandrasudevan

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1.University Database – Normalization (Up to 3NF)

```
mysql> SELECT * FROM Student;
+-----+-----+-----+
| StudentID | StudentName | Department |
+-----+-----+-----+
|          101 | Alice       | CSE        |
|          102 | Bob        | ECE        |
+-----+-----+-----+
2 rows in set (0.066 sec)
```

```
mysql> SELECT * FROM Faculty;
+-----+-----+
| FacultyID | FacultyName |
+-----+-----+
|          1 | Dr. Kumar   |
|          2 | Dr. Priya   |
+-----+-----+
2 rows in set (0.031 sec)
```

```
mysql> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | FacultyID |
+-----+-----+-----+
|          201 | DBMS       |          1 |
|          202 | Python     |          2 |
+-----+-----+-----+
2 rows in set (0.009 sec)
```

```
mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| StudentID | CourseID | Grade |
+-----+-----+-----+
|      101 |      201 | A     |
|      101 |      202 | B+    |
|      102 |      201 | A-    |
+-----+-----+-----+
3 rows in set (0.009 sec)
```

```
mysql> SELECT
->     s.StudentID,
->     s.StudentName,
->     s.Department,
->     c.CourseName,
->     f.FacultyName,
->     e.Grade
-> FROM Student s
-> JOIN Enrollment e ON s.StudentID = e.StudentID
-> JOIN Course c ON e.CourseID = c.CourseID
-> JOIN Faculty f ON c.FacultyID = f.FacultyID;
+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | Department | CourseName | FacultyName | Grade |
+-----+-----+-----+-----+-----+-----+
|      101 | Alice       | CSE        | DBMS       | Dr. Kumar   | A     |
|      101 | Alice       | CSE        | Python     | Dr. Priya   | B+    |
|      102 | Bob         | ECE        | DBMS       | Dr. Kumar   | A-    |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.540 sec)
```

2. Employees working in multiple departments

```
mysql> SELECT
->     e.EmpID,
->     e.EmpName,
->     COUNT(ed.DeptID) AS TotalDepartments
-> FROM Employee e
-> JOIN EmployeeDepartment ed
-> ON e.EmpID = ed.EmpID
-> GROUP BY e.EmpID, e.EmpName
-> HAVING COUNT(ed.DeptID) > 1;
```

EmpID	EmpName	TotalDepartments
101	Alice	2
104	David	2

2 rows in set (0.300 sec)

```
mysql> SELECT
->     e.EmpName,
->     d.DeptName
-> FROM Employee e
-> JOIN EmployeeDepartment ed
-> ON e.EmpID = ed.EmpID
-> JOIN Department d
-> ON ed.DeptID = d.DeptID
-> WHERE e.EmpID IN (
->     SELECT EmpID
->     FROM EmployeeDepartment
->     GROUP BY EmpID
->     HAVING COUNT(DeptID) > 1
-> )
-> ORDER BY e.EmpName;
```

EmpName	DeptName
Alice	HR
Alice	Finance
David	HR
David	IT

4 rows in set (0.113 sec)

3.BCNF Decomposition

```
mysql> INSERT INTO StudentCourseInfo VALUES
-> ('S101','C101','Dr. Kumar','9876543210'),
-> ('S101','C102','Dr. Priya','9876501234'),
-> ('S102','C101','Dr. Kumar','9876543210'),
-> ('S103','C103','Dr. Ravi','9876511111');
```

Query OK, 4 rows affected (0.357 sec)
Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM StudentCourseInfo;
```

StudentID	CourseID	Instructor	InstructorPhone
S101	C101	Dr. Kumar	9876543210
S101	C102	Dr. Priya	9876501234
S102	C101	Dr. Kumar	9876543210
S103	C103	Dr. Ravi	9876511111

4 rows in set (0.018 sec)

```
mysql> CREATE TABLE StudentCourse (
-> StudentID VARCHAR(10),
-> CourseID VARCHAR(10),
-> Instructor VARCHAR(50),
-> PRIMARY KEY (StudentID, CourseID),
-> FOREIGN KEY (Instructor) REFERENCES Instructor(Instructor)
-> );
```

Query OK, 0 rows affected (1.285 sec)

```
mysql> SELECT * FROM Instructor;
```

Instructor	InstructorPhone
Dr. Kumar	9876543210
Dr. Priya	9876501234
Dr. Ravi	9876511111

3 rows in set (0.026 sec)

```
mysql> SELECT
-> sc.StudentID,
-> sc.CourseID,
-> sc.Instructor,
-> i.InstructorPhone
-> FROM StudentCourse sc
-> JOIN Instructor i
-> ON sc.Instructor = i.Instructor;
```

StudentID	CourseID	Instructor	InstructorPhone
S101	C101	Dr. Kumar	9876543210
S102	C101	Dr. Kumar	9876543210
S101	C102	Dr. Priya	9876501234
S103	C103	Dr. Ravi	9876511111

4 rows in set (0.042 sec)

4. Banking System – Views and Constraints

Query OK, 4 rows affected (0.177 sec)
Records: 4 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Customer;
```

CustomerID	CustomerName	AccountType	Balance	Branch
101	Rahul	Savings	25000.00	Chennai
102	Priya	Current	50000.00	Coimbatore
103	Arun	Savings	15000.00	Madurai
104	Sneha	Current	80000.00	Salem

4 rows in set (0.048 sec)

Query OK, 0 rows affected (0.342 sec)

```
mysql> SELECT * FROM StaffView;
```

CustomerID	CustomerName	AccountType	Branch
101	Rahul	Savings	Chennai
102	Priya	Current	Coimbatore
103	Arun	Savings	Madurai
104	Sneha	Current	Salem

4 rows in set (0.111 sec)

Query OK, 0 rows affected (0.169 sec)

```
mysql> SELECT * FROM ManagerView;
```

CustomerID	CustomerName	AccountType	Balance	Branch
101	Rahul	Savings	25000.00	Chennai
102	Priya	Current	50000.00	Coimbatore
103	Arun	Savings	15000.00	Madurai
104	Sneha	Current	80000.00	Salem

4 rows in set (0.015 sec)

5. Sales Database – Normalization (3NF)

Query OK, 4 rows affected (0.267 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM SalesInfo;

SaleID	CustomerID	CustomerName	ProductID	ProductName	SupplierID	SupplierName	Quantity
1	101	Rahul	201	Laptop	301	Dell	2
2	101	Rahul	202	Mouse	302	Logitech	1
3	102	Priya	201	Laptop	301	Dell	1
4	103	Arun	203	Keyboard	302	Logitech	3

4 rows in set (0.007 sec)

Query OK, 4 rows affected (0.132 sec)

Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Customer;

CustomerID	CustomerName
101	Rahul
102	Priya
103	Arun

3 rows in set (0.006 sec)

mysql> ^C

mysql> SELECT * FROM Supplier;

SupplierID	SupplierName
301	Dell
302	Logitech

2 rows in set (0.005 sec)

mysql> SELECT * FROM Product;

ProductID	ProductName	SupplierID
201	Laptop	301
202	Mouse	302
203	Keyboard	302

3 rows in set (0.006 sec)


```
mysql> SELECT * FROM Sales;
```

SaleID	CustomerID	ProductID	Quantity
1	101	201	2
2	101	202	1
3	102	201	1
4	103	203	3

```
4 rows in set (0.016 sec)
```

```
mysql> SELECT
```

```
-> s.SaleID,  
-> c.CustomerName,  
-> p.ProductName,  
-> su.SupplierName,  
-> s.Quantity  
-> FROM Sales s  
-> JOIN Customer c ON s.CustomerID = c.CustomerID  
-> JOIN Product p ON s.ProductID = p.ProductID  
-> JOIN Supplier su ON p.SupplierID = su.SupplierID;
```

SaleID	CustomerName	ProductName	SupplierName	Quantity
1	Rahul	Laptop	Dell	2
3	Priya	Laptop	Dell	1
2	Rahul	Mouse	Logitech	1
4	Arun	Keyboard	Logitech	3

```
4 rows in set (0.077 sec)
```

6. Customers and Orders – Joins & Subqueries

```

> ON c.CustomerID = o.CustomerID,
+-----+-----+-----+
| CustomerID | CustomerName | OrderID | Amount |
+-----+-----+-----+
|          101 | Rahul        |         1 | 1500.00 |
|          101 | Rahul        |         2 | 2500.00 |
|          101 | Rahul        |         4 | 1800.00 |
|          102 | Priya        |         3 | 3000.00 |
|          103 | Arun         |         5 | 1200.00 |
|          104 | Sneha        |         6 | 5000.00 |
|          104 | Sneha        |         7 | 2000.00 |
+-----+-----+-----+
7 rows in set (0.098 sec)

```

```

> HAVING COUNT(o.OrderID) > 1,
+-----+-----+-----+
| CustomerID | CustomerName | TotalOrders |
+-----+-----+-----+
|          101 | Rahul        |            3 |
|          104 | Sneha        |            2 |
+-----+-----+-----+
2 rows in set (0.077 sec)

```

```

-> );
+-----+-----+-----+
| CustomerID | CustomerName | City        |
+-----+-----+-----+
|          101 | Rahul        | Chennai    |
|          104 | Sneha        | Salem     |
+-----+-----+-----+
2 rows in set (0.036 sec)

```

```

> GROUP BY c.CustomerName,
+-----+-----+
| CustomerName | TotalAmount |
+-----+-----+
| Rahul        | 5800.00     |
| Priya        | 3000.00     |
| Arun         | 1200.00     |
| Sneha        | 7000.00     |
+-----+-----+
4 rows in set (0.012 sec)

```

7. Multi-Valued Dependencies – Decompose into 4NF

```
mysql> SELECT * FROM StudentInfo;
+-----+-----+-----+
| StudentID | Hobby   | Language |
+-----+-----+-----+
|          101 | Cricket | English  |
|          101 | Cricket | Tamil    |
|          101 | Music   | English  |
|          101 | Music   | Tamil    |
|          102 | Reading | English  |
|          102 | Reading | Hindi    |
+-----+-----+-----+
6 rows in set (0.007 sec)
```

```
mysql> SELECT * FROM StudentHobby;
+-----+-----+
| StudentID | Hobby   |
+-----+-----+
|          101 | Cricket |
|          101 | Music   |
|          102 | Reading |
+-----+-----+
3 rows in set (0.006 sec)
```

```
mysql> SELECT * FROM StudentLanguage;
+-----+-----+
| StudentID | Language |
+-----+-----+
|          101 | English  |
|          101 | Tamil    |
|          102 | English  |
|          102 | Hindi    |
+-----+-----+
4 rows in set (0.007 sec)
```

8.Integrity Constraints – Evaluate Schema

```
mysql> SELECT * FROM Employee;
```

EmpID	EmpName
101	Alice
102	Bob
103	Charlie
104	David

```
4 rows in set (0.062 sec)
```

```
mysql> INSERT INTO Employee VALUES  
-> (101,'Kiran','kiran@gmail.com',26,30000,'Sales');
```

```
mysql> INSERT INTO Employee VALUES  
-> (104,'Vijay','rahul@gmail.com',27,38000,'Marketing');
```

```
mysql> INSERT INTO Employee VALUES  
-> (105,'Ajay','ajay@gmail.com',16,25000,'HR');
```

```
mysql> INSERT INTO Employee VALUES  
-> (106,'Manoj','manoj@gmail.com',25,-5000,'IT');
```

```
mysql> INSERT INTO Employee  
-> (EmpID,EmpName,Email,Age,Salary)  
-> VALUES  
-> (107,'Ravi','ravi@gmail.com',26,30000);
```

9.Join Dependencies – Decompose into 5NF

```
mysql> SELECT * FROM Supply;
```

SupplierID	ProductID	ProjectID
1	101	1001
1	102	1001
2	101	1002
2	103	1002

```
4 rows in set (0.006 sec)
```

```
mysql> SELECT * FROM SupplierProduct;
```

SupplierID	ProductID
1	101
1	102
2	101
2	103

```
4 rows in set (0.006 sec)
```

```
mysql> SELECT * FROM ProductProject;
```

ProductID	ProjectID
101	1001
101	1002
102	1001
103	1002

```
4 rows in set (0.005 sec)
```

```
mysql> SELECT * FROM SupplierProject;
```

SupplierID	ProjectID
1	1001
2	1002

```
2 rows in set (0.008 sec)
```

10. Optimize SQL Queries

```
mysql> SELECT DISTINCT  
-> c.CustomerID,  
-> c.CustomerName,  
-> o.OrderID,  
-> o.Amount  
-> FROM Customer c  
-> JOIN Orders o  
-> ON c.CustomerID = o.CustomerID;
```

CustomerID	CustomerName	OrderID	Amount
101	Rahul	1	1500.00
101	Rahul	2	2500.00
102	Priya	3	3000.00
103	Arun	4	1800.00
104	Sneha	5	5000.00

```
5 rows in set (0.025 sec)
```

```
-> Table scan on <temporary> (cost=4.06..6.11 rows=5)
```

```
-> Temporary table with deduplication (cost=3.55..3.55 rows=5)
```

```
-> Nested loop inner join (cost=2.4 rows=5)
```

```
-> Table scan on c (cost=0.65 rows=4)
```

```
-> Index lookup on o using idx_customerid (CustomerID = c.CustomerID) (cost=0.344 rows=1.25)
```